DistriCine: An Object-Oriented Prototype for University Film Club Management

Diego Alejandro Yáñez Zabala, Brayan Sebastián Díaz Ramírez, and Oscar Javier Camargo Trujillo
Systems Engineering Program
Universidad Distrital Francisco José de Caldas
Bogotá, Colombia
Email: {20251020103, 20251020150, 20251020143}@udistrital.edu.co

***Abstract*—University film clubs often suffer from low student engagement due to reliance on manual, non-digital processes for scheduling and promotion. To address this, we propose *DistriCine*, a lightweight, object-oriented application designed specifically for academic cinema management across multiple campuses. Our solution implements core functionalities—movie catalog browsing, schedule consultation, and reservation handling—using Java and foundational Object-Oriented Programming (OOP) principles. The prototype demonstrates a modular, role-based architecture that successfully executes all primary use cases in a console environment, validating both functional correctness and design coherence while laying the groundwork for future graphical and persistent extensions.**

***Index Terms*—Object-Oriented Programming, University Film Club, Role-Based Access, Java, UML, Cultural Technology**

I. Introduction

Cultural initiatives such as university film clubs are essential for fostering critical thinking, artistic appreciation, and community building among students. However, at institutions like the Universidad Distrital Francisco José de Caldas, these activities are typically coordinated through informal channels—posters, classroom announcements, or word of mouth—leading to poor visibility, inefficient planning, and low attendance. Commercial cinema platforms (e.g., CineMark, Eventbrite) are unsuitable for this context due to their complexity, cost, and lack of integration with academic workflows such as multi-campus operations or student identification systems.

Previous academic efforts have explored event management systems using object-oriented design [**?**], but few focus specifically on the cultural domain of university film screenings. Moreover, many prototypes prioritize interface aesthetics over architectural soundness, resulting in systems that are difficult to maintain or extend. In contrast, our work emphasizes *design integrity* through rigorous application of OOP principles—encapsulation, inheritance, polymorphism, and composition—as the foundation for a scalable solution.

This paper presents *DistriCine*, a console-based prototype developed to bridge the gap between cultural need and technical execution. Unlike generic ticketing tools, DistriCine is tailored to the university context, supporting role-based access (student vs. administrator), campus-specific scheduling, and modular expansion. By grounding the system in object-oriented methodology and validating it through functional testing, we demonstrate that a well-structured core can precede—and enable—future enhancements like web interfaces or database integration.

II. Methods and Materials

The development of DistriCine followed an iterative object-oriented design process, beginning with requirement elicitation and culminating in a validated Java prototype. Functional requirements included displaying movie listings with metadata (title, genre, duration, rating, synopsis), filtering by genre, consulting screening schedules by campus, and managing reservations. Non-functional requirements emphasized Windows compatibility, sub-3-second response time, and interface simplicity.

We adopted a *role-based user model* to reflect real-world responsibilities. An abstract `User` class defines common attributes (name, email, password) and declares an abstract `showMenu()` method. Two concrete subclasses—`CustomerUser` (for students) and `AdminUser` (for coordinators)—override this method to present context-specific options, such as "Make Reservation" or "Add Movie." This design leverages *inheritance* for code reuse and *polymorphism* for behavioral differentiation, adhering to the Liskov Substitution Principle.

Data integrity is ensured through *encapsulation*: all class attributes are private, with access mediated by public getters and setters. For example, the `Movie` class stores immutable metadata initialized via constructor, preventing inconsistent states. Similarly, the `Reservation` class composes a `Movie` instance with date and time strings, modeling a "has-a" relationship that reflects real-world semantics.

The system architecture is strictly modular. Core classes include `User`, `CustomerUser`, `AdminUser`, `Movie`, and `Reservation`. No external libraries or frameworks were used, ensuring portability and clarity. Although a graphical interface was prototyped in Figma (see Fig. 1), implementation was deferred to prioritize *business logic validation* in a console environment—a deliberate choice to verify architectural soundness before UI investment.

Assumptions include in-memory data persistence (no database), static movie catalogs during runtime, and simulated authentication (hardcoded credentials). These limitations were

accepted to maintain focus on OOP fidelity and reproducibility within academic constraints.

Fig. 1: Figma prototype of DistriCine's main screen, showing movie listings and featured banner. (Note: GUI not implemented in current prototype.)

III. Results and Discussion

The prototype was validated through functional execution in a Java 17 console environment. All core use cases—user login simulation, role-specific menu display, movie creation, and reservation confirmation—executed as expected. A representative test sequence (see Listing 1) demonstrates polymorphism in action: calling `showMenu()` on both user types yields distinct outputs without conditional logic.

Unit testing focused on object behavior rather than edge cases, given the prototype scope. Each class was verified for correct initialization, proper encapsulation, and role-specific behavior. Integration was validated through object collaboration: a `Reservation` correctly references a `Movie` and prints its title via `getTitle()`, confirming composition integrity.

Acceptance criteria from user stories (e.g., "filter by genre") are supported structurally—the `Movie` class includes a `genre` attribute, enabling future filtering logic without redesign. While quantitative metrics (e.g., response time) were not instrumented, manual timing confirmed compliance with the ¡3-second NFR.

Compared to ad-hoc scripting approaches, DistriCine's OOP foundation offers superior maintainability and extensibility. For instance, adding a new user role (e.g., "Guest") requires only a new subclass—no modification to existing code—demonstrating the Open/Closed Principle in practice.

```
Customer Menu:
1. View schedule
2. Make a reservation
3. Check history
Administrator Menu:
1. Add Movie
2. Edit Schedule
3. Manage Venues
Reservation confirmed for Inception on
    2025-10-25 at 19:00
```

Listing 1: Sample execution output (simplified and Idealized)

IV. Conclusions

DistriCine successfully demonstrates that a well-structured object-oriented core can effectively model the domain of university film club management. By prioritizing encapsulation, inheritance, and polymorphism, we created a system that is not only functional but also pedagogically valuable and technically extensible. The console prototype validates all essential workflows—user differentiation, movie display, and reservation handling—proving that architectural rigor can precede graphical polish.

This work contributes a reproducible, scoped solution to a real cultural challenge in academia. Future efforts will focus on implementing the Figma-designed web interface, integrating a relational database for persistence, and connecting to university authentication systems for single sign-on. Additional features like seat selection, trailer embedding, and recommendation engines are also planned. Ultimately, DistriCine aims to transform how universities engage students with cinema—not as passive spectators, but as active participants in a digitally empowered cultural ecosystem.

Acknowledgment

The authors thank Ing. Carlos Andrés Sierra, M.Sc., for his guidance in the Object-Oriented Programming course at Universidad Distrital Francisco José de Caldas.

References